

DA3408 (AIOps) - Assignment 2

Sujal Jayant Kumar

Roll No. - DA24B026

Q1

Size	Naive	Multi-stage
Image size	862 MB	591 MB

The Multistage image is much smaller (31.44 % times), few things are left out in it. The naive builds it without the ‘-no-cache-dir’ so all the packages are downloaded and stay in image, including pandas which is not a requirement for ‘multi-stage’ because none of the copied files in multi-stage use it. In multi-stage the python packages are not cached and only the virtual env is copied with two more files `main.py` and `model.joblib` while in naive, it copied everything from the `app/` directory.

Q2

A single Dockerfile can create one image, so lets say 2 images depend on each other for a function to happen, it cannot cooperating process between them. Compose solves this using an `yaml` file and can have multi-container definitions.

Kubernetes works around different machines, like it cooperates and works around different machines, making/rescheduling jobs and updates while compose works on one machine and it cannot function if the other images are on some other machine and not the same one.

Request	Latency
First (cache miss)	0.0669s
Second (cache hit)	0.0048s

Q3

1.

One pod would at max require one core. My chosen parallelism is 4 and completions is 8 because there are 8 shards. Each node has 2000m allocatable, which must also cover Kubernetes’ own system pods, so at 1000m per pod, two pods would use node’s entire core without leaving anything for Kubernetes’ own system. I therefore set CPU requests/limits to 800m, so two pods reserve 1600m of 2000m and leave ≈ 400 m of headroom - two pods fit per node, and across 2 nodes that gives my chosen parallelism of 4. For the Downward API I expose two fields with `valueFrom.fieldRef`, since neither can be hardcoded: `POD_NAME` from `metadata.name` and `NODE_NAME` from `spec.nodeName`. The worker prints both into its result line, so each shard’s count is attributable to a specific pod on a specific node.

3.

Shard	0	1	2	3	4	5	6	7	Total
Invalid rows	15	5	11	9	6	10	13	6	75

A shared volume would have not worked because minikube's default storage provisioner is local to a single node and binds a volume to one node and my 8 pods ran across 2 nodes, provisioner would just get the directory one host system (one node) and skip the other node. Thus half of the pods could not have reached the logs.

4.

I would not change my parallelism choice. When parallelism is 4 and completion is 8, it runs the tasks twice to cover all completions, and all cores are used both times. But if its 6, the second run would only need to run across 2 shards, which means 4/6 cores would sit idle. The time is still the same, but now you are allowing more concurrency without any benefit. Another thing can be to increase it to 8 but it would be bad as well because then you have to allot $8 \times 800m = 6400m$ for a 6000m (6 cores) job and 2 pods will still be pending while the other 6 complete their work, so the parallelism won't be 8.

Q4

2.

The ReplicaSet controller is responsible for the self-healing process. In my `yaml` file `replicas: 2` and `selector.matchLabels: app: spam-api` are the ones that control the ReplicaSet, the selector knows now, which pods to count and how many should be there. If the pods are not equal to the number of replicas then ReplicaSet will just create a replacement from the `template` block.

3.

A job will not have a rolling update mechanism, there is no controller to move up the newer version while the older version moves down. The way out would be to delete and recreate the job, leaving the API unreachable in this process. A job expects its pods to execute and terminate and it counts it as completions, but this deployment runs forever, and it would never mark it as completed but as soon as pod exits, it would just mark it as completed rather than replacing it.

4.

A deployment keeps running while treating exits as some error and recreates the pods instantly after deleting. While a Job treats a pod exiting successfully as the goal, having completion in its `yaml` file. Deployments have rolling update, if you update your image, the rollout replaces them without downtime, but if you want to update a job you have to delete and rerun it. The spam-API task has no end state, its a running model, it should be an API on a stable address while validating shards is just a task than needs to be done with concurrent cores.